2026-10-23 MySQL的内置函数

- 1. 日期与时间函数
  - 1.1 先区分日期、时间和日期时间
  - 1.2 获取当前日期时间
  - 1.3 日期加减与间隔单位
  - 1.4 计算两个日期相差多少天
  - 1.5 案例一：创建生日表并插入当前日期
  - 1.6 案例二：查询最近两分钟发布的帖子
- 2. 字符串函数
  - 2.1 查看字符集与拼接字符串
  - 2.2 查找子串
  - 2.3 生成考生成绩文字说明
  - 2.4 LENGTH() 与 CHAR_LENGTH()：字节数和字符数
  - 2.5 替换、截取与大小写转换
  - 2.6 去除空格
- 3. 数学函数
  - 3.1 绝对值与进制转换
  - 3.2 向上取整与向下取整
  - 3.3 保留小数位：FORMAT()、ROUND() 与 TRUNCATE()
  - 3.4 随机数
  - 3.5 取模与负数
- 4. 其他常用函数
  - 4.1 查询当前用户和当前数据库
  - 4.2 MD5() 摘要
  - 4.3 PASSWORD() 的版本与安全说明
  - 4.4 使用 IFNULL() 处理空值
- 5. 实战与易错点
  - 5.1 实战题：统计逗号出现次数
  - 5.2 常见易错点
  - 5.3 组合使用函数的方法
- 6. 一页速查
  - 日期时间
  - 字符串
  - 数学与其他函数
- 总结

内置函数是 SQL 中可以直接调用的数据处理工具。它们既可以单独用于查询和测试，也可以嵌套在 `WHERE`、`SELECT`、`INSERT` 等语句中，让数据库完成日期计算、字符串整理、数值转换和空值处理。

> **学习目标**
>
> 能够根据业务问题选择合适的函数；理解日期运算、字符长度、取整和 `NULL` 等易错点；能够把函数组合起来解决真实的查询问题。

内容导航

1. 日期与时间函数
2. 字符串函数
3. 数学函数
4. 其他常用函数
5. 实战与易错点
6. 一页速查

1. 日期与时间函数

1.1 先区分日期、时间和日期时间

- `DATE`：日期，格式类似 `2026-10-23`，表示年月日。
- `TIME`：时间，格式类似 `14:30:25`，表示时分秒。
- `DATETIME`：日期和时间，格式类似 `2026-10-23 14:30:25`。
- `TIMESTAMP`：带时区语义的时间类型；实际显示结果与 MySQL 会话时区有关。

MySQL 函数名不区分大小写，`NOW()` 与 `now()` 等价。为了阅读清晰，本文统一使用大写函数名。

函数	返回值	典型用途	关键点
`CURRENT_DATE()`	当前日期	获取年月日	可省略括号，写成 `CURRENT_DATE`
`CURRENT_TIME()`	当前时间	获取时分秒	可省略括号，写成 `CURRENT_TIME`
`CURRENT_TIMESTAMP()`	当前日期时间	获取当前完整时间	结果类似 `2026-10-23 14:30:25`

函数	返回值	典型用途	关键点
`NOW()`	当前日期时间	插入或比较当前时间	与 `CURRENT_TIMESTAMP()` 等价
`DATE(expr)`	日期部分	从日期时间中截取日期	`DATE(NOW())` 得到今天的日期
`DATE_ADD(date, INTERVAL n unit)`	计算后的日期时间	在日期或时间上增加一段间隔	单位排在数值之后
`DATE_SUB(date, INTERVAL n unit)`	计算后的日期时间	在日期或时间上减去一段间隔	常用于“最近 N 分钟”查询
`DATEDIFF(date1, date2)`	相差天数	计算两个日期相差多少天	只按日期部分计算，结果为 `date1 - date2`

注意：`CURRENT_TIMESTAMP()` 的含义是“当前日期和时间”，不是 Unix 时间戳。如果需要 Unix 秒数，应使用 `UNIX_TIMESTAMP()`。

1.2 获取当前日期时间

```
SELECT CURRENT_DATE() AS today;
SELECT CURRENT_TIME() AS current_time;
SELECT CURRENT_TIMESTAMP() AS current_datetime;
SELECT NOW() AS now_value;
```

`NOW()` 返回日期和时间，`CURRENT_DATE()` 只返回日期部分。也可以通过函数嵌套完成转换：

```
SELECT DATE(NOW()) AS today;
SELECT DATE('1949-10-01 00:00:00') AS founding_date;
```

`DATE()` 的作用不是简单截断字符串，而是从日期时间值中提取日期部分。

1.3 日期加减与间隔单位

```
SELECT DATE_ADD('2024-01-01', INTERVAL 40 DAY) AS date_after_40_days;

SELECT DATE_ADD(NOW(), INTERVAL 10 SECOND) AS after_10_seconds;
SELECT DATE_ADD(NOW(), INTERVAL 10 MINUTE) AS after_10_minutes;

SELECT DATE_SUB('2017-10-01', INTERVAL 2 DAY) AS date_before_2_days;
```

`INTERVAL` 的基本形式是：

```
INTERVAL 数值 单位
```

常用单位包括 `SECOND`、`MINUTE`、`HOUR`、`DAY`、`WEEK`、`MONTH`、`YEAR`。函数名和单位是 SQL 关键字，不需要加引号；数值直接写在 `INTERVAL` 后面。

1.4 计算两个日期相差多少天

```
SELECT DATEDIFF('2017-10-10', '2016-09-01') AS days_diff;
SELECT DATEDIFF(NOW(), '1949-10-01') AS days_since_founding;
```

`DATEDIFF(date1, date2)` 的结果是 `date1 - date2`：

- 前面的日期更晚，结果为正数。
- 前面的日期更早，结果为负数。
- 它只计算“相差多少天”，不能直接告诉你有几个小时的差别。

如果业务需要更精确的时间差，可以使用以下补充写法：

```
SELECT TIMESTAMPDIFF(MINUTE, '2026-10-23 09:00:00', '2026-10-23 09:45:00')
AS minutes_diff;
```

1.5 案例一：创建生日表并插入当前日期

```
CREATE TABLE tmp (
    id INT PRIMARY KEY AUTO_INCREMENT,
    birthday DATE
);

INSERT INTO tmp (birthday) VALUES ('1991-01-01');
INSERT INTO tmp (birthday) VALUES (CURRENT_DATE());

SELECT * FROM tmp;
```

这里演示了两种数据来源：固定日期和数据库当前日期。字段类型为 `DATE`，因此只保存年月日。

1.6 案例二：查询最近两分钟发布的帖子

先创建一张留言表：

```
CREATE TABLE msg (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    content VARCHAR(100) NOT NULL,
    sendtime DATETIME
);
```

插入一条使用当前时间作为发送时间的评论：

```
INSERT INTO msg (content, sendtime)
VALUES ('哈哈哈哈哈哈哈哈哈哈哈', NOW());
```

运行结果如下：

```
mysql> select * from msg;
+----+-------------------------------------+---------------------+
| id | content                             | sendtime            |
+----+-------------------------------------+---------------------+
|  1 | 哈哈哈哈哈哈哈哈哈哈哈              | 2026-10-22 19:10:16 |
+----+-------------------------------------+---------------------+
1 row in set (0.00 sec)
```

现在查询两分钟以内发布的帖子。判断条件是“发送时间晚于两分钟前的时刻”：

```
SELECT content, sendtime
FROM msg
WHERE sendtime > DATE_SUB(NOW(), INTERVAL 2 MINUTE);
```

课件中的等价写法是：

```
SELECT *
FROM msg
WHERE DATE_ADD(sendtime, INTERVAL 2 MINUTE) > NOW();
```

两种写法表达的是同一个时间区间。工程实践中更推荐前一种写法，因为右侧的 `DATE_SUB(NOW(), ...)` 可以在查询开始时算出一个固定边界，而 `DATE_ADD(sendtime,`

`...)` 对每一行都进行计算，通常不利于普通索引进行范围扫描。

`>` 表示严格晚于两分钟前；如果要包含恰好两分钟前的记录，应使用 `>=`。

如果只想显示发布日期而不显示时分秒：

```
SELECT content, DATE(sendtime) AS send_date
FROM msg;
```

2. 字符串函数

字符串函数用于处理编码、拼接、查找、大小写转换、截取和替换。使用时要特别区分“字符数”和“字节数”，并注意 SQL 字符串下标从 `1` 开始。

函数	作用	关键规则
`CHARSET(str)`	返回字符串使用的字符集	用于查看连接的字符编码
`CONCAT(str2, ...)`	拼接多个字符串	任意一个参数为 `NULL` 时，结果为 `NULL`
`INSTR(string, substring)`	查找子串首次出现的位置	从 `1` 开始；找不到返回 `0`
`UCASE(str)` / `UPPER(str)`	转换为大写	二者作用相同
`LCASE(str)` / `LOWER(str)`	转换为小写	二者作用相同
`LEFT(str, length)`	从左取指定长度的字符	长度表示字符数量
`LENGTH(str)`	返回字符串字节数	中文在 `utf8mb4` 中通常占 3 或 4 个字节
`CHAR_LENGTH(str)`	返回字符串字符数	更适合与“有几个字”相关的问题
`REPLACE(str, search_str, replace_str)`	替换全部匹配内容	不是只替换第一次出现的位置
`STRCMP(str1, str2)`	比较两个字符串	相等返回 `0`，按排序规则返回 `-1` 或 `1`
`SUBSTRING(str, position [, length])`	从指定位置取子串	位置从 `1` 开始
`LTRIM(str)`	去掉左侧空格	不处理中间空格
`RTRIM(str)`	去掉右侧空格	不处理中间空格
`TRIM(str)`	去掉左右两侧空格	不处理中间空格

2.1 查看字符集与拼接字符串

```sql
SELECT CHARSET('abcd') AS string_charset;
SELECT CONCAT('a', 'b', 'c') AS result;
SELECT CHARSET(ename) FROM emp;
```

`CHARSET()` 返回的是字符集名称，例如 `utf8mb4`。它不代表单个字符占用了多少字节。

拼接时还要注意 `NULL`：

```sql
SELECT CONCAT('A', NULL, 'B') AS result;
```

结果为 `NULL`。如果希望把缺失值替换成空字符串，可以组合 `IFNULL()`：

```sql
SELECT CONCAT('A', IFNULL(NULL, ''), 'B') AS result;
```

2.2 查找子串

```sql
SELECT INSTR('asdfg1245', '1234ghj') AS position;
```

因为 `'asdfg1245'` 中不存在完整的子串 `'1234ghj'`，所以结果为 `0`。如果存在，则返回第一次出现时首字符的位置。

```sql
SELECT INSTR('MySQL', 'SQL') AS position; -- 3
```

2.3 生成考生成绩文字说明

```sql
SELECT CONCAT(
    '考生姓名：', name,
    '，总分：', chinese + math + english,
    '，语文成绩：', chinese,
    '，数学成绩：', math,
    '，英语成绩：', english
) AS msg
FROM exam_result;
```

这个例子展示了两类组合：

- `CONCAT()` 负责把文字和字段值拼接成一段说明。
- `chinese + math + english` 先完成数值计算，再把结果作为参数交给 `CONCAT()`。

如果任一科成绩为 `NULL`，直接相加的结果也会是 `NULL`。业务上若要把缺考按 `0` 分处理，可以写 `IFNULL(chinese, 0) + IFNULL(math, 0) + IFNULL(english, 0)`。

2.4 `LENGTH()` 与 `CHAR_LENGTH()`：字节数和字符数

```
SELECT name, LENGTH(name) AS bytes_count
FROM exam_result;
```

运行结果：

```
mysql> select name,length(name) from exam_result;
+-----------+--------------+
| name      | length(name) |
+-----------+--------------+
| 唐三藏    |            9 |
| 猪悟能    |            9 |
| 曹孟德    |            9 |
| 孙权      |            6 |
| 宋公明    |            9 |
+-----------+--------------+
5 rows in set (0.00 sec)
```

`LENGTH()` 统计的是字节数，不是汉字或字母的个数。在常见 `utf8mb4` 配置下，一个汉字通常占 3 个字节，因此“唐三藏”的 `LENGTH()` 为 `9`，但 `CHAR_LENGTH()` 为 `3`。

```
SELECT
    name,
    CHAR_LENGTH(name) AS characters_count,
    LENGTH(name) AS bytes_count
FROM exam_result;
```

选择规则：

- 问“名字有几个字”，使用 `CHAR_LENGTH()`。
- 问“存储占多少字节”，使用 `LENGTH()`。

2.5 替换、截取与大小写转换

将 `EMP` 表姓名中的大写 `S` 全部替换成“上海”：

```
SELECT ename, REPLACE(ename, 'S', '上海') AS replaced_name
FROM emp;
```

截取 `ename` 字段的第二个到第三个字符：

```
SELECT SUBSTRING(ename, 2, 2) AS name_part, ename
FROM emp;
```

`SUBSTRING(ename, 2, 2)` 的含义是“从第 2 个字符开始，取 2 个字符”。MySQL 的字符串位置从 `1` 开始，不是从 `0` 开始。

将姓名的首字母转成小写，再与其余部分拼接：

```
SELECT
    CONCAT(LCASE(SUBSTRING(ename, 1, 1)), SUBSTRING(ename, 2)) AS lower_name
FROM emp;
```

可以按“先截取、再转换、最后拼接”的顺序理解：

1. `SUBSTRING(ename, 1, 1)` 取出首字母。
2. `LCASE(...)` 把首字母转成小写。
3. `SUBSTRING(ename, 2)` 取出剩余部分。
4. `CONCAT(...)` 把两部分重新拼接。

再补充几个短示例：

```
SELECT LEFT('MySQL', 2) AS first_two;          -- My
SELECT UPPER('mysql') AS uppercase_value;      -- MYSQL
SELECT STRCMP('abc', 'abd') AS compare_result; -- -1
```

2.6 去除空格

```sql
SELECT LTRIM('      你好    ') AS left_trimmed;
SELECT RTRIM('      你好    ') AS right_trimmed;
SELECT TRIM('      你好    ') AS both_trimmed;
```

- `LTRIM()` 只去掉左侧空格。
- `RTRIM()` 只去掉右侧空格。
- `TRIM()` 同时去掉左右两侧空格。
- 三种函数都不去除字符串中间的空格。

如果业务要求立即删除全部空格，应使用替换：

```sql
SELECT REPLACE('  你 好  ', ' ', '') AS no_spaces;
```

3. 数学函数

数学函数用于绝对值、进制转换、取整、格式化、随机数和取模。最容易出错的是负数的取整方向，以及格式化结果的数据类型。

函数	作用	示例结果
`ABS(number)`	求绝对值	`ABS(-100.2) = 100.2`
`BIN(decimal_number)`	十进制转二进制	`BIN(10) = '1010'`
`HEX(decimal_number)`	十进制转十六进制	`HEX(255) = 'FF'`
`CONV(number, from_base, to_base)`	在指定进制之间转换	`CONV('FF', 16, 10) = '255'`
`CEILING(number)` / `CEIL(number)`	向上取整	`CEILING(23.04) = 24`
`FLOOR(number)`	向下取整	`FLOOR(23.7) = 23`
`FORMAT(number, decimal_places)`	格式化并保留小数位	`FORMAT(12.3456, 2) = '12.35'`
`RAND()`	返回 `[0, 1)` 范围内的随机浮点数	例如 `0.734...`
`MOD(number, denominator)`	取模、求余	`MOD(10, 3) = 1`

3.1 绝对值与进制转换

```sql
SELECT ABS(-100.2) AS absolute_value;
SELECT BIN(10) AS binary_text;
```

```sql
SELECT HEX(255) AS hexadecimal_text;
SELECT CONV('FF', 16, 10) AS decimal_text;
```

`BIN()` 和 `HEX()` 返回的是字符串形式，而不是供继续做十进制算术的数值。

3.2 向上取整与向下取整

```sql
SELECT CEILING(23.04) AS ceil_value; -- 24
SELECT FLOOR(23.7) AS floor_value;   -- 23
```

对负数要特别小心：

```sql
SELECT FLOOR(-1.2) AS floor_negative;    -- -2
SELECT CEILING(-1.2) AS ceil_negative;   -- -1
SELECT TRUNCATE(-1.2, 0) AS toward_zero; -- -1
```

`FLOOR()` 永远是向数轴上更小的方向取整，所以 `FLOOR(-1.2)` 是 `-2`。原笔记里“负数直接去掉小数部分”的说法容易被理解成向零取整，而向零截断应使用 `TRUNCATE(number, 0)`。

3.3 保留小数位：`FORMAT()`、`ROUND()` 与 `TRUNCATE()`

```sql
SELECT FORMAT(12.3456, 2) AS formatted_value;
```

结果为 `'12.35'`，`FORMAT()` 会四舍五入到指定小数位，并返回格式化后的字符串；数值较大时还可能出现千位分隔符。

```sql
SELECT ROUND(12.3456, 2) AS rounded_number;
SELECT TRUNCATE(12.3456, 2) AS truncated_number;
```

- `ROUND()`：按指定精度四舍五入，结果仍是数值。
- `TRUNCATE()`：直接截去多余小数位，不进行四舍五入，结果仍是数值。
- `FORMAT()`：面向展示的格式化，结果应优先按字符串对待。

3.4 随机数

```sql
SELECT RAND() AS random_value;
```

`RAND()` 返回大于等于 `0`、小于 `1` 的浮点数。生成 `0` 到 `99` 的随机整数，可以写成：

```sql
SELECT FLOOR(RAND() * 100) AS random_integer;
```

3.5 取模与负数

```sql
SELECT MOD(10, 3) AS remainder;
SELECT MOD(-10, 3) AS negative_remainder;
```

MySQL 中取模结果的符号通常与被除数一致，因此 `MOD(-10, 3)` 为 `-1`。不要把“负数取模”简单理解成一定返回正数。

4. 其他常用函数

4.1 查询当前用户和当前数据库

```sql
SELECT USER() AS current_user;
SELECT DATABASE() AS current_database;
```

- `USER()` 返回当前 MySQL 连接使用的账户，通常会包含用户名和来源主机。
- `DATABASE()` 返回当前默认数据库；如果尚未选择数据库，结果为 `NULL`。

4.2 `MD5()` 摘要

```sql
SELECT MD5('admin') AS password_digest;
```

结果是一个长度为 32 的十六进制字符串：

```
21232f297a57a5a743894a0e4a801fc3
```

`MD5()` 的特点是输入长度不影响输出长度，相同的输入会得到相同的摘要。因此，它常被用于文件校验或数据指纹，但不能把它当作安全可靠的密码存储方案。

4.3 `PASSWORD()` 的版本与安全说明

示例为：

```
SELECT PASSWORD('root');
```

该函数用于 MySQL 早期版本的账户密码散列。`PASSWORD()` 在 MySQL 5.7 中已废弃，并在 MySQL 8.0 中移除；现代项目不应依赖它。

另外，直接使用无盐 `MD5()` 保存密码也不安全，因为相同密码会得到相同摘要，而且 MD5 已不适合作为密码哈希算法。实际应用应在业务后端使用带盐、可调计算成本的密码哈希方案，例如 bcrypt、scrypt 或 Argon2。

4.4 使用 `IFNULL()` 处理空值

`IFNULL(value1, value2)` 的规则是：

- `value1` 为 `NULL` 时，返回 `value2`。
- `value1` 不为 `NULL` 时，返回 `value1`。

```
SELECT IFNULL('abc', '123') AS first_result;
SELECT IFNULL(NULL, '123') AS second_result;
```

结果分别是 `abc` 和 `123`。它只判断 `NULL`，空字符串 `''` 仍然算有值：

```
SELECT IFNULL('', '默认值') AS empty_string_result;
```

结果是空字符串，不会返回“默认值”。

需要多个候选值时，可以使用补充函数 `COALESCE()`：

```
SELECT COALESCE(NULL, NULL, '后备值') AS first_non_null;
```

`COALESCE()` 返回第一个非 `NULL` 参数，适合处理多个可能为空的字段。

5. 实战与易错点

5.1 实战题：统计逗号出现次数

题目：统计字符串 `'10,A,B'` 中逗号 `','` 出现的次数。

思路是“原字符串长度减去删除逗号后的长度”，因为每次替换都会删除一个逗号：

```
SELECT
    CHAR_LENGTH('10,A,B') -
    CHAR_LENGTH(REPLACE('10,A,B', ',', '')) AS cnt;
```

结果为 `2`。推荐使用 `CHAR_LENGTH()` 而不是 `LENGTH()`，这样即使待统计的分隔符是中文等多字节字符，也能按字符数量正确比较：

```
SELECT
    CHAR_LENGTH('数据库，MySQL，函数') -
    CHAR_LENGTH(REPLACE('数据库，MySQL，函数', '，', '')) AS cnt;
```

5.2 常见易错点

易错点	正确理解
把 `CURRENT_TIMESTAMP()` 当成 Unix 时间戳	它返回当前日期时间；Unix 秒数用 `UNIX_TIMESTAMP()`
混淆 `NOW()` 和 `CURRENT_DATE()`	`NOW()` 包含时分秒，`CURRENT_DATE()` 只有年月日
`DATEDIFF()` 参数顺序反了	结果是前者减后者，应确认正负号代表的业务含义
以为 `DATEDIFF()` 能精确计算小时差	它只返回日期之差；精细差值使用 `TIMESTAMPDIFF()`
用 `LENGTH()` 统计字符数	`LENGTH()` 返回字节数；字符数使用 `CHAR_LENGTH()`
以为 `SUBSTRING()` 从 `0` 开始	MySQL 的字符串位置从 `1` 开始
以为 `TRIM()` 会删除所有空格	它只删除左右两侧空格，中间空格保留
以为 `FLOOR()` 等于向零取整	`FLOOR()` 向负无穷方向取整；向零截断用 `TRUNCATE()`
以为任意负数取模都为正	取模结果符号通常与被除数一致
把 `IFNULL()` 当成空字符串判断	它只将 `NULL` 视为缺失，空字符串不算 `NULL`
用 MD5 或 `PASSWORD()` 保存现代用户密码	它们不适合作为现代密码存储方案

5.3 组合使用函数的方法

复杂查询通常不是寻找一个“万能函数”，而是把多个简单函数嵌套起来：

```
DATE(NOW())
→ 先取得当前日期时间，再取日期部分

CONCAT(LCASE(SUBSTRING(ename, 1, 1)), SUBSTRING(ename, 2))
→ 截取首字母 → 转小写 → 截取其余部分 → 拼接

CHAR_LENGTH(str) - CHAR_LENGTH(REPLACE(str, ',', ''))
→ 统计原长度 → 删除目标字符后统计长度 → 两者相减
```

阅读嵌套函数时，可以由内向外逐层分析；编写时则先确认每一步的输入和输出类型，避免把字符串、数值和日期类型混在一起。

6. 一页速查

日期时间

```
CURRENT_DATE()                          -- 当前日期
CURRENT_TIME()                          -- 当前时间
CURRENT_TIMESTAMP() / NOW()             -- 当前日期时间
DATE(expr)                              -- 提取日期部分
DATE_ADD(expr, INTERVAL n UNIT)         -- 增加时间
DATE_SUB(expr, INTERVAL n UNIT)         -- 减少时间
DATEDIFF(date1, date2)                  -- 日期相差天数
TIMESTAMPDIFF(unit, start, end)         -- 更精细的时间差
```

字符串

```
CHARSET(str)                            -- 字符集
CONCAT(...)                             -- 拼接
INSTR(str, substring)                   -- 子串位置，找不到为 0
UCASE(str) / UPPER(str)                 -- 转大写
LCASE(str) / LOWER(str)                 -- 转小写
LEFT(str, length)                       -- 左侧截取
LENGTH(str)                             -- 字节数
CHAR_LENGTH(str)                        -- 字符数
REPLACE(str, old, new)                  -- 全部替换
STRCMP(str1, str2)                      -- 字符串比较
SUBSTRING(str, position, length)        -- 指定位置截取
LTRIM(str) / RTRIM(str) / TRIM(str) -- 去左右空格
```

数学与其他函数

```
ABS(number)                          -- 绝对值
BIN(number)                          -- 二进制字符串
HEX(number)                          -- 十六进制字符串
CONV(number, from_base, to_base)     -- 进制转换
CEILING(number)                      -- 向上取整
FLOOR(number)                        -- 向下取整
ROUND(number, precision)             -- 四舍五入
TRUNCATE(number, precision)          -- 截断小数
FORMAT(number, precision)            -- 格式化展示
RAND()                               -- [0, 1) 随机数
MOD(number, denominator)             -- 取模
USER()                               -- 当前用户
DATABASE()                           -- 当前数据库
MD5(str)                             -- MD5 摘要，不适合作密码存储
IFNULL(value1, value2)               -- NULL 时使用后备值
COALESCE(value1, value2, ...)        -- 返回第一个非 NULL 值
```

总结

这份内容的组织主线不是死记函数名，而是把函数放回具体问题中：

- 处理时间：先判断需要日期、时间还是完整日期时间，再确认加减单位和差值粒度。
- 处理文本：先区分字符与字节，再确认查找位置从 `1` 开始，以及是否允许 `NULL`。
- 处理数值：关注取整方向、格式化和负数取余。
- 处理缺失值：`IFNULL()` 与 `COALESCE()` 负责兜底，但要明确 `NULL` 和空字符串不是一回事。

掌握这些规则后，就可以把多个函数组合起来，完成留言时间筛选、成绩说明生成、姓名整理和字符串计数等常见任务。